

Movie Reservation System: Project Instruction Breakdown

Introduction

Building a Movie Reservation System is an advanced backend project that encompasses relational data modeling, transaction management, scheduling, and payment integration. This guide breaks down the development process from scratch into manageable, actionable steps.

Step 1: Requirements Analysis & Data Modeling

Objective: Define the core entities and their relationships.

Tasks: - **Identify Entities:** - User: ID, Name, Email, Password, etc. - Movie: ID, Title, Description, Genre, Duration, Release Date. - Theater: ID, Name, Location. - TheaterRoom: ID, TheaterID, Name, Capacity. - Seat: ID, RoomID, Row, Column, Type (e.g., VIP, Regular). - Schedule (Showtime): ID, MovieID, RoomID, StartTime, EndTime. - Reservation: ID, UserID, ScheduleID, TotalPrice, Status, PaymentReference. - ReservedSeat: ID, ReservationID, SeatID. - **Design ER Diagram:** Map the 1-to-many and many-to-many relationships (e.g., A TheaterRoom has many Seats, a Reservation has many ReservedSeats). - **Choose Database:** Select a relational database (e.g., PostgreSQL or MySQL) to handle complex transactions and ACID properties.

Step 2: Project Setup & Environment Configuration

Objective: Initialize the backend environment and connect to the database.

Tasks: - **Initialize Project:** Choose a backend framework (e.g., Node.js/Express, Python/Django/FastAPI, Java/Spring Boot) and set up the repository. - **Install Dependencies:** Setup an ORM (e.g., Prisma, TypeORM, SQLAlchemy, Hibernate) for database interactions. - **Database Setup:** Configure connection strings and run initial migrations to create the tables designed in Step 1. - **Environment Variables:** Create a .env file for database URIs, port configurations, and secret keys.

Step 3: Core API - Movies, Theaters, and Schedules

Objective: Build CRUD (Create, Read, Update, Delete) operations for the foundational data.

Tasks: - **Movies Endpoint:** - GET /movies: List movies with filters for dates, genres, and actors. - POST /movies (Admin): Add a new movie. - **Theaters & Rooms Endpoint:** Manage physical locations and the rooms inside them. - **Schedules Endpoint:** - Create endpoints to assign a movie to a specific TheaterRoom at a specific time. - Ensure logic checks for overlapping schedules in the same room.

Step 4: Virtual Theater & Seating Logic

Objective: Represent the theater layout and track seat availability for specific showtimes.

Tasks: - **Seat Generation:** Write a script or endpoint to auto-generate Seat records for a TheaterRoom based on row and column configurations. - **Availability Endpoint:** - GET /schedules/{id}/seats: Return all seats for a specific showtime. - Cross-reference Seat data with ReservedSeat data for that schedule to mark seats as available, reserved, or locked.

Step 5: Reservation & Transaction Management

Objective: Handle the core booking flow securely, avoiding double-booking.

Tasks: - **Seat Locking Mechanism:** - When a user selects seats, temporarily "lock" them (e.g., for

10 minutes) using a caching layer like Redis or a timestamp in the database. - **Booking Endpoint:** - POST /reservations: Accept UserID, ScheduleID, and an array of SeatIDs. - **Transaction/Concurrency Control:** - Use database transactions (e.g., Serializable or Read Committed isolation levels). - Use pessimistic locking (SELECT . . . FOR UPDATE) or optimistic locking to ensure two users cannot book the same seat simultaneously.

Step 6: Payment Integration (Stripe)

Objective: Process payments securely before finalizing the reservation.

Tasks: - **Stripe Setup:** Register for a Stripe developer account and get API keys. - **Checkout Session:** - When the user confirms seats, create a Stripe PaymentIntent or Checkout Session. - Return the client secret to the frontend. - **Webhooks:** - Implement a secure webhook endpoint (e.g., POST /webhooks/stripe) to receive payment success or failure events. - On success: Update the Reservation status to "Confirmed". - On failure/timeout: Release the locked seats.

Step 7: Search, Filtering & Final Polish

Objective: Refine the user experience and ensure system robustness.

Tasks: - **Advanced Search:** Implement robust queries to allow filtering movies by showtime, genre, and location. - **Authentication/Authorization:** Secure endpoints (e.g., only admins can add movies; users can only see their own reservations) using JWT or sessions. - **Error Handling & Validation:** Ensure all inputs are sanitized and descriptive errors are returned for edge cases (e.g., "Seat already booked"). - **Testing:** Write unit and integration tests for the reservation transaction logic.

Conclusion

This Movie Reservation System touches on essential backend concepts: relational data, concurrency, third-party integrations, and complex business logic. Completing this step-by-step will result in a robust, portfolio-ready application.